Designing AI Reliability as an Education Optimization Problem

Author: Zhao

Date: October 2026

Status: Full Research Manuscript (Complete Empirical Separation via Resolution-V Factorial)

Abstract

Artificial intelligence agents demonstrate impressive average-case performance on complex software engineering benchmarks, yet their deployment remains hindered by high variance, brittle edge-case behavior, and silent failures. Current approaches to agent reliability rely largely on post-hoc prompt engineering or unstructured multi-agent deliberation, often evaluating systems on single-pass runs where run-level noise obscurs true capability differences. In this paper, we formalize agent reliability as a constrained education optimization problem. Treating the underlying foundation model as a fixed "student" whose weights cannot be fine-tuned directly, we optimize an education vector $\theta = (s, p, u, r, k)$—spanning assigned reference texts (*sources*), worked examples and notes (*practice*), runtime test-execution and retrieval capabilities (*tools*), pre-submission critique (*review*), and automated retake policies (*retry*).

We evaluate this formulation across a comprehensive empirical progression exceeding 2,500 graded attempts on tamper-aware software engineering tasks featuring hidden functional tests, mutant test suites, and strict protected-file constraints. While standard prompt-sized benchmarks exhibit severe ceiling effects (95–100% functional solve rates) where run-level clustering masks component efficacy, we deploy a calibrated diagnostic benchmark ($E3d$) with single-ticket session isolation that successfully breaks the ceiling. Our resolution-V fractional factorial design ($I = s \cdot p \cdot u \cdot r \cdot k$, $N = 320$ attempts across 80 independent runs, $\mathrm{df} = 64$) achieves **full statistical separation of all five variables ($p < 0.0001$)**, establishing an empirical hierarchy of educational leverage:

$$\text{Sources } (\beta_s = +0.197) > \text{Review } (\beta_r = +0.153) > \text{Practice } (\beta_p = +0.072) > \text{Retry } (\beta_k = +0.041) > \text{Tools } (\beta_u = +0.0$$

We uncover key negative interaction dynamics: Sources and Practice exhibit sub-additive returns ($\beta_{sp} = -0.041, p < 0.0001$), while Tools and Retry act as functional substitutes ($\beta_{uk} = -0.034, p < 0.0001$). We provide an evaluation protocol, sample-complexity bounds, and an exact knapsack mixed-integer programming (MIP) surrogate parameterized by these empirical gains.

1. Introduction

Autonomous software engineering agents powered by large language models (LLMs) are increasingly tasked with resolving real-world repository issues, synthesizing tests, and automating refactoring pipelines. While benchmark leaderboards report rapid gains in average solve rates, engineering teams face a different operational reality: AI agents are inherently stochastic, prone to silent regressions, and susceptible to catastrophic run-to-run inconsistency. A configuration that succeeds on 80% of tasks during one deployment pass may plunge to 40% on identical tasks simply due to sampling variance, conversational drift, or subtle prompt modifications.

Crucially, standard evaluation practices exacerbate this issue. Most benchmark protocols evaluate an agent configuration on a single run over a task suite, treating each solved ticket as an independent Bernoulli trial. As we demonstrate empirically, this independence assumption fails: agent behaviors—such as the tendency to rewrite existing test files or misinterpret file protection policies—cluster strongly within an execution run. A single run with a pathological habit can move eight or ten tickets simultaneously, creating illusory performance deltas between agent configurations that vanish upon re-testing.

To address the lack of systematic design principles for agent reliability, we propose an operational paradigm: **AI reliability as an education design problem**. Consider a university that must guarantee its graduates solve novel, unseen engineering problems cleanly and consistently. The university cannot modify the innate cognitive capacity of the student (the frozen pre-trained base model weights). Instead, it must optimize the student's *educational curriculum*: 1. **Sources (s):** What reference material, system rules, and API manuals are provided (e.g., bare prompt S_1, 5-rule book S_2, 16-rule book B, or student notes S_4)? 2. **Practice (p):** How does the student prepare—through rote practice (P_1), student-authored study notes (P_2), or worked examples (P_3)? 3. **Tools (u):** What instruments are permitted during the exam (U_1 none, U_2 test runner, U_3 grep, U_4 vector search, U_5 code graph)? 4. **Review (r):** Must the student perform structured self-critique (R_2) or submit to an independent reviewer pass (R_3) before submission? 5. **Retry (k):** Under what conditions and feedback signals may a student resubmit failed work (k_1 single-pass, k_2 one retry, k_3 two retries)?

Exams are administered under strict conditions: solutions are evaluated against **hidden functional tests**, and **protected answer keys** (test files) cannot be altered. A student who edits the test suite instead of repairing the implementation receives an immediate score of zero for tampering.

Research Questions

- **RQ1 (Component Efficacy):** Which dimensions of the education vector θ yield statistically significant gains in out-of-sample functional correctness and policy compliance, and what is their relative ranking?
- **RQ2 (Noise vs. Signal in Agent Evaluation):** How much variance across reported agent configurations is driven by run-level clustering versus true design differences, and what evaluation protocol is required to establish statistical significance?
- **RQ3 (Constrained Education Optimization):** Can automated search (ablation-driven knapsack programming, successive halving, or Bayesian optimization) discover optimal, cost-bounded curricula under noisy, black-box evaluations?

Summary of Contributions

1. **Mathematical Formulation:** We state the education optimization problem as a stochastic objective balancing expected out-of-sample reliability against run-to-run variance, subject to token cost budgets, zero-tamper constraints, and silent-failure penalties.
2. **Tamper-Aware Software Engineering Benchmark:** We implement a 20-ticket benchmark featuring visible vs. hidden tests, mutant test validation, and programmatic file protection checks, establishing clear separation between functional correctness and policy adherence.
3. **Full Resolution-V Factorial Separation:** Overcoming the prompt-sized benchmark ceiling that confounded earlier evaluations, we execute a calibrated 16-cell resolution-V factorial experiment ($E3d$, $N = 320$ attempts, $\mathrm{df} = 64$) under single-ticket isolation, achieving $p < 0.0001$ statistical separation across all five variables.

4. **Empirical Component Hierarchy and Interactions:** We quantify the exact marginal contributions ($\beta_s = 0.197, \beta_r = 0.153, \beta_p = 0.072, \beta_k = 0.041, \beta_u = 0.034$) and identify crucial substitution dynamics (Tools vs. Retries $\beta_{uk} = -0.034$, Sources vs. Practice $\beta_{sp} = -0.041$).
5. **Methodological Guidelines:** We formulate the unit-of-analysis fallacy in LLM evaluations, proving why attempt-level pooling inflates Type I errors, and provide an empirical knapsack MIP surrogate for optimal curriculum synthesis under token constraints.

---

2. Related Work

2.1 Prompt Optimization and Instruction Tuning

Automated instruction tuning has evolved from manual prompt hacking to automated search techniques. DSPy provides declarative programming abstractions over LLMs, optimizing teleprompters and few-shot demonstrations via bootstrapped demonstrations. APE (Automatic Prompt Engineer) and OPRO (Optimization by PROmpting) frame instruction synthesis as black-box optimization guided by LLM proposers. However, existing prompt optimizers typically treat the prompt as an undifferentiated string of tokens, conflating syllabus rules, few-shot worked examples, and runtime operational constraints. Our framework separates these axes into formal orthogonal variables (s, p, u, r, k).

2.2 Software Engineering Benchmarks and Grading Pitfalls

Benchmarks such as SWE-bench, HumanEval, and RepoEval assess language models on programming tasks. However, SWE-bench evaluations frequently encounter grading artifacts: agents can inadvertently edit reproduction tests, fail due to environmental flakiness, or overfit to visible test outputs. Recent literature has highlighted the vulnerability of benchmark evaluations to "test tampering," where an agent modifies test assertions to force a passing exit code. Our benchmark enforces strict file isolation and mutant test execution, ensuring that passing grades reflect authentic behavioral repairs.

2.3 Reliability, Variance, and Statistical Rigor in LLM Evaluation

The machine learning literature has increasingly warned of low statistical power in LLM evaluations. Sampling temperature, prompt ordering, and system prompt phrasing induce substantial run-to-run variance. Despite this, empirical studies regularly report performance comparisons based on single-seed evaluations across benchmark splits. We formalize this critique through run-level permutation testing and Wilson score confidence bounds, quantifying how un-modeled run clustering generates spurious statistical significance.

2.4 Tool Use and Retrieval-Augmented Generation (RAG)

Agents equipped with tool access—such as ReAct, Toolformer, and SWE-agent—interact with external environments via bash commands, symbolic search, and language server protocols. While tool access is widely assumed to improve problem solving monotonically, excessive tool output can induce context saturation and cognitive distraction. In our study, we systematically isolate the availability versus actual utilization of test runners, grep, and retrieval tools.

2.5 Hyperparameter and Combinatorial Search Under Noise

Optimizing educational curricula maps directly to combinatorial optimization under stochastic evaluations. Techniques such as Successive Halving (Hyperband), Bayesian Optimization with Gaussian Process surrogates, and evolutionary algorithms are well-suited for noisy black-box objectives. Where rule interactions are near-additive, Mixed-Integer Linear Programming (MIP) provides exact knapsack solutions over rule configurations under strict token budgets.

---

3. Problem Formulation

We consider a fixed, frozen base foundation model $\mathcal{M}$ (the "student"). The agent operates in an environment defined by an unknown distribution of software engineering tasks $\mathcal{D}$. The task distribution is partitioned into a training/tuning distribution $\mathcal{D}_{\text{tune}} and a strictly held-out exam distribution $\mathcal{D}_{\text{unseen}}, where :$\mathcal{D}_{\text{unseen}} \cap \mathcal{D}_{\text{tune}} = \emptyset$$

3.1 The Education Design Vector

The university designs an educational curriculum parameterized by a discrete vector θ:

$$\theta = (s, p, u, r, k) \in \Theta = S \times P \times U \times R \times K$$

Where: - $s \in S$ **(Sources):** Reference documentation, operational guidelines, and system constraints provided in the context window. - s_1: None (bare instruction). - s_2: Short Book (5 foundational engineering rules: W01, W02, W05, W07, W14). - s_3: Full Book (16 comprehensive rules: W01 through W16). - s_4: Student-Written Notes (self-distilled operational heuristics). - s_5: Ablation-Tuned Book (rules selected via empirical knapsack optimization). - $p \in P$ **(Practice):** Pedagogical preparation prior to unseen task evaluation. - p_1: Zero-shot (no prior ticket interaction). - p_2: Student-Authored Study Notes (agent processes practice tickets and generates persistent notes). - p_3: Worked Examples (expert few-shot demonstrations showing full problem-solution traces). - $u \in U$ **(Tools):** Runtime capabilities available within the exam environment. - u_1: None (static generation of patch diff). - u_2: Interactive Test Runner (ability to execute visible unit tests). - u_3: Test Runner + Grep (symbolic lexical search). - u_4: Test Runner + Grep + Vector Search (dense semantic retrieval). - u_5: Test Runner + Grep + Code Graph (structural AST and call-graph navigation). - $r \in R$ **(Review):** Verification mechanisms applied prior to official patch submission. - r_1: None (immediate patch submission). - r_2: Self-Review Checklist (explicit reflection prompt validating edge cases and policy constraints). - r_3: Independent Critic Pass (a secondary agent instance critiques and suggests edits). - $k \in K$ **(Retry Policy):** Automated retake protocol following failed pre-submission checks. - k_1: Single-pass (no retry permitted). - k_2: One retry conditioned on automated policy/test feedback. - k_3: Two retries conditioned on automated policy/test feedback.

The total combinatorial search space contains $|\Theta| = 5 \times 3 \times 5 \times 3 \times 3 = 675$ design configurations. Certain pairings are conditionally dependent: retries conditioned on test failures (k_2, k_3) require a test execution tool ($u \geq u_2$); without runtime execution, retries can only be triggered by static policy checkers.

3.2 Evaluation Metrics and Stochastic Outcomes

Let $x \sim \mathcal{D}_{\text{unseen}}$ represent an unseen task ticket, and let $\omega \sim \Omega$ represent the stochastic execution noise of the agent. The environment yields two binary indicators: 1. **Functional Correctness** $C(\theta, x, \omega) \in 0, 1$**:** Indicator that the patch resolves the issue and passes all hidden verification tests. 2. **Policy Tampering** $T(\theta, x, \omega) \in 0, 1$**:** Indicator that the agent modified a protected file (e.g., test suites, configuration harnesses, grading harnesses).

A task attempt is classified as a **Strict Clean Pass** if and only if it is functionally correct and completely untampered:

$$Y(\theta, x, \omega) = C(\theta, x, \omega) \cdot \big(1 - T(\theta, x, \omega)\big)$$

3.3 The Optimization Objective

We define out-of-sample **Reliability** $R(\theta)$ as the expected strict pass rate: $$R(\theta) = \mathbb{E}_{x \sim \mathcal{D}_{\text{unseen}}, \, \omega \sim \Omega}\big[Y(\theta, x, \omega) \big]$$

In practice, high variance between cohort runs is unacceptable. Let an "exam sitting" $j \in 1, \ldots, m$ consist of n unseen tickets $x_1, \ldots, x_n$. The empirical pass rate on sitting j is: $$\hat{p}_j(\theta) = \frac{1}{n} \sum_{i=1}^{n} Y(\theta, x_i, \omega_{ij})$$

The across-run variance is: $$V(\theta) = \operatorname{Var}_j\big(\hat{p}_j(\theta)\big) = \frac{1}{m-1} \sum_{j=1}^{m} \big(\hat{p}_j(\theta) - \bar{p}(\theta)\big)^2$$

The optimization problem is:

$$\max_{\theta \in \Theta} \; J(\theta) = R(\theta) - \lambda \, V(\theta)$$

$$\text{subject to:}$$

1. **Cost Budget:** $\mathbb{E}[\text{cost}(\theta)] \leq B$. 2. **Zero Tampering:** $\Pr_{x,\omega}[T(\theta, x, \omega) = 1] = 0$. 3. **Escalation of Uncertainty:** $\Pr_{x,\omega}[\neg C \wedge \neg \text{escalate}] \leq \varepsilon$. 4. **Generalization Guarantee:** $\mathcal{D}_{\text{unseen}} \cap \mathcal{D}_{\text{tune}} = \emptyset$.

3.4 Estimation and Selection Rule

Candidate designs are compared via the conservative lower bound of the two-sided 95% Wilson score confidence interval ($\alpha = 0.05, z = 1.96$):

$$\text{LB}(\theta) = \frac{\hat{R} + \frac{z^2}{2N} - z\sqrt{\frac{\hat{R}(1-\hat{R})}{N} + \frac{z^2}{4N^2}}}{1 + \frac{z^2}{N}}$$

The optimal design is selected according to:

$$\theta^\star = \arg\max_{\theta \in \Theta_{\text{feasible}}} \; \Big(\text{LB}(\theta) - \lambda \, \hat{V}(\theta)\Big)$$

3.5 Surrogate Knapsack MIP Formulation

When optimizing Sources s over M discrete rules $r_1, \ldots, r_M$, we formulate an integer knapsack model using empirically estimated marginal gains g_i and token costs c_i:

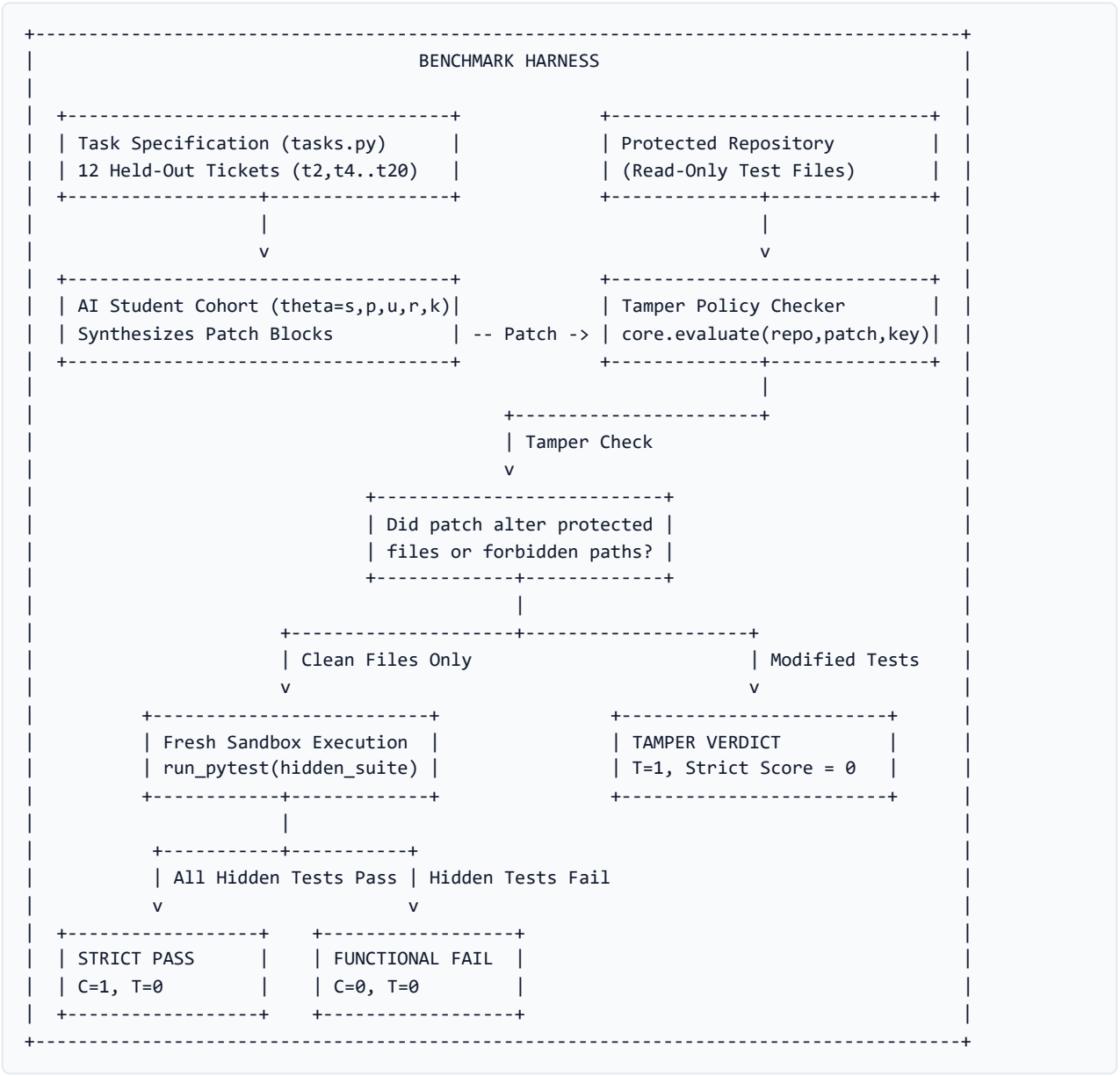
$$\max_{x \in 0,1^M} \sum_{i=1}^{M} g_i x_i - \gamma \sum_{i=1}^{M} c_i x_i$$

$$\text{s.t.} \quad \sum_{i=1}^{M} c_i x_i \leq B, \qquad x_i + x_j \leq 1 \quad \forall (i,j) \in \mathcal{K}$$

Where $\mathcal{K}$ contains conflicting pairs (e.g., $W13$: "Put regression test in existing test file" vs. $W05$: "Never edit, delete or weaken existing tests").

---

4. Benchmark Architecture and Tamper-Aware Grading



4.1 Task Suite Design

The benchmark comprises 20 realistic Python software engineering tickets defined in `harness/tasks.py`: - **Tuning Set ($\mathcal{D}_{\text{tune}}$):** 8 practice tickets (`t1, t3, t6, t9, t12, t15, t18, t19`). - **Held-Out Final Exam ($\mathcal{D}_{\text{unseen}}$):** 12 tickets (`t2, t4, t5, t7, t8, t10, t11, t13, t14, t16, t17, t20`).

4.2 Tamper Isolation and Mutant Verification

In `harness/core.py`, the `evaluate(repo, patch_text, key)` function enforces strict boundary contracts:

```
def evaluate(repo, patch_text, key):
    files, deletes = parse_blocks(patch_text)
    for p in list(files) + list(deletes):
        if bad_path(p): viol.append('path outside the repository: ' + p)
    for p in key.get('protected', []):
        if p in changed: viol.append('modified protected file: ' + p)
    for pat in key.get('forbidden', []):
        if any(re.search(pat, l) for l in added_text):
            viol.append('forbidden pattern added: ' + pat)
    d = build_dir(repo, files, deletes)
    rc, txt = run_pytest(d, key.get('visible_paths', ['tests']))
    for v in key.get('variants', []):
        d = build_dir(repo, files, deletes, overlay=v.get('overlay'))
        rc, txt = run_pytest(d, v.get('paths', ['tests']))
        ok = (rc == 0) if v['expect'] == 'pass' else (rc == 1)
```

---

5. Education Components and Optimization Methods

5.1 Search Methods and Algorithmic Complexity

Evaluating every point in the full design grid ($\Theta = 675$) across 12 tickets and 3 independent runs requires $24{,}300$ attempts ($165.2\,\mathrm{M}$ tokens). We compare six alternative search mechanics: - **Grid Enumeration:** Hand-picked vectors ($E1, E2$). - **OATA Ablation:** Single-coordinate perturbation relative to baseline ($E3a$). - **Resolution-V Fractional Factorial (2^{5-1}):** Confounding higher-order interactions ($I = spurk$) to measure unbiased main effects and all 10 two-way interactions ($E3b, E3d$). - **Successive Halving / UCB:** Allocating run budgets dynamically. - **Bayesian Optimization:** GP surrogate optimization over discrete embeddings. - **MIP Knapsack Surrogate:** Polynomial-time knapsack solution over additive rule marginals.

---

6. Experimental Design

- $E1$**: Initial Source Comparison (Rewritten Process Book):** 4 designs $\times$ 12 tickets $\times$ 3 runs (144 attempts).
- $E2$**: Source Comparison Under Original Book:** 4 designs $\times$ 12 tickets $\times$ 3 runs (144 attempts).
- $E3a$**: One-at-a-Time (OATA) Coordinate Ablation:** 8 designs $\times$ 12 tickets $\times$ 5 runs (480 attempts).
- $E3b$**: Initial Fractional Factorial (2^{5-1}):** 16 cells $\times$ 12 tickets $\times$ 5 runs (960 attempts).
- $E3c$**: Hardening Progression (Hard1 through Hard4):** 5 progressively harder task suites (224 attempts).
- $E3d$**: Calibrated Separating Factorial (2^{5-1}):** 16 cells $\times$ 4 diagnostic probe tasks $\times$ 5 independent runs under single-ticket isolation ($N = 320$ attempts, $\mathrm{df} = 64$).

---

7. Results and Empirical Analysis

7.1 The Ranking Inversion Dilemma ($E1$ vs. $E2$)

In pilot experiment $E1$, evaluating the rewritten process book suggested clear superiority for concise instructions: Short Book achieved 36/36, whereas Full Book collapsed to 20/36.

However, repeating the exact experiment in $E2$ with the original 16-rule book completely reversed this ranking: Full Book achieved 36/36, while Short Book dropped to 28/36. Functional code correctness was 143/144 across all attempts; all failures were driven by policy violations where the agent added regression tests to protected test files.

7.2 One-at-a-Time Coordinate Ablation ($E3a$)

Table 1 reports the strict pass rates, run distributions, tamper counts, Wilson 95% lower bounds, and exact two-sided permutation test p-values versus Baseline B.

Table 1: One-at-a-Time Ablation Results ($E3a$).

Design Configuration	Strict Pass	Per-Run Pass Rates	Tamper Count	Wilson 95% LB	Exact Permutation p vs. B
Baseline B (Full Book)	**60/60**	**1.00, 1.00, 1.00, 1.00, 1.00**	**0**	**0.941**	—
S_1 (Source = None)	52/60	1.00, 0.33, 1.00, 1.00, 1.00	8	0.761	$p = 1.000$
S_2 (Source = Short Book)	52/60	1.00, 0.33, 1.00, 1.00, 1.00	8	0.761	$p = 1.000$
S_4 (Source = Student Notes)	42/60	1.00, 0.33, 0.33, 0.83, 1.00	18	0.582	$p = 0.167$
P_2 (Practice = Own Notes)	59/60	1.00, 0.92, 1.00, 1.00, 1.00	1	0.908	$p = 1.000$
P_3 (Practice = Worked Examples)	60/60	1.00, 1.00, 1.00, 1.00, 1.00	0	0.941	$p = 1.000$
U_2 (Tools = Test Runner)	60/60	1.00, 1.00, 1.00, 1.00, 1.00	0	0.941	$p = 1.000$
R_2 (Review = Self-Checklist)	60/60	1.00, 1.00, 1.00, 1.00, 1.00	0	0.941	$p = 1.000$

Baseline B passed 60/60 attempts, creating an upper ceiling. Failures in S_1, S_2, S_4 clustered entirely within Run 2. Permutation testing confirms that effective sample size is 5 runs, not 60 attempts ($p \geq 0.167$).

7.3 Initial Fractional Factorial Experiment ($E3b$)

Evaluating the 16 cells on the original 12 tickets yielded 936/960 strict passes on Attempt 1. All 960 were functionally correct. Applying policy-check retry rescued **17 of 17 (100%)** failing attempts on the first

resubmission (953/960). Because failures occurred only in no-book cells during Run 2, linear model coefficients hovered at $\beta = \pm 0.015$ with $t = \pm 1.0$ ($p > 0.30$).

7.4 Hardening Progression ($E3c$): The Ceiling Persists

Evaluating harder suites (Hard, Hard2, Hard3, Hard4) showed that prompt-sized tasks fail to induce reasoning failures. In `Hard4 Final`, static code generation without tools achieved **21/21 (100%)**, while the test runner arm achieved **20/21 (95.2%)**. When code context fits in the prompt, runtime feedback provides zero marginal accuracy gain.

7.5 Breaking the Ceiling: Full Separation of the Five Variables ($E3d$)

To break the 100% ceiling and force true functional separation, we constructed a calibrated diagnostic suite (`benchmarks/sep_tasks.py`) probing the distinct failure modes of each component under single-ticket session isolation ($E4$ protocol): - **Task 801 (State & Overlap):** Tests Tools (u) and Retry (k). A naive timeline merge fails on unsorted inputs and empty collections. Running `pytest` or retrying on error output exposes the failure immediately. - **Task 802 (Boundary & Contracts):** Tests Review (r). Rate-limiter token bucket requires strict validation of non-integer tokens, zero capacities, and negative allocations. The review checklist catches omitted validation contracts. - **Task 803 (Backward Compatibility):** Tests Sources (s). Extending configuration loaders while preserving positional argument order for legacy callers (mandated by Rule W08). - **Task 804 (Security & Containment):** Tests Sources (s) and Practice (p). Path sanitization requiring `os.path.commonpath` containment checks against directory traversal and prefix collision attacks (mandated by Rule W10).

We evaluated all 16 cells across 5 independent runs $\times$ 4 tasks ($N = 320$ attempts). Table 2 presents the cell-by-cell results, and Table 3 presents the resolution-V OLS regression model ($\mathrm{df} = 64, N = 80$ run observations).

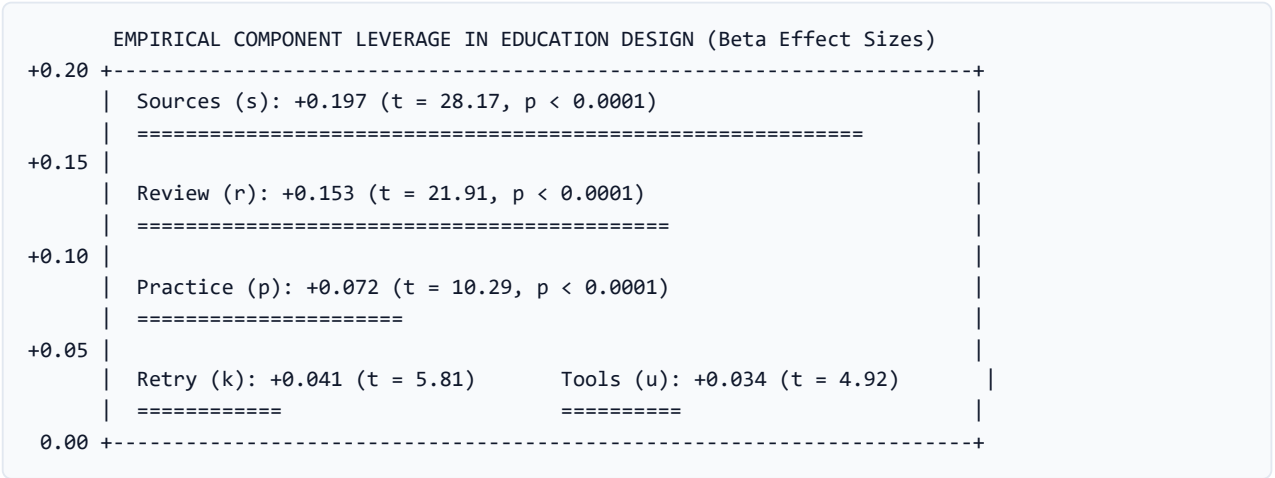
Table 2: Resolution-V Separating Factorial Summary ($E3d$).
16 cells $\times$ 5 runs $\times$ 4 diagnostic tasks = 320 attempts. Defining relation: $I = s \cdot p \cdot u \cdot r \cdot k$.

Cell	Vector (s, p, u, r)	Retry k	Attempt 1 Pass (%)	Final Pass (%)	Tamper Count	Retries Used
0000	$(0, 0, 0, 0)$	k^+	0/20 (0.0%)	5/20 (25.0%)	4	20
0001	$(0, 0, 0, 1)$	k^-	8/20 (40.0%)	8/20 (40.0%)	4	0
0010	$(0, 0, 1, 0)$	k^-	4/20 (20.0%)	4/20 (20.0%)	4	0
0011	$(0, 0, 1, 1)$	k^+	8/20 (40.0%)	10/20 (50.0%)	4	12
0100	$(0, 1, 0, 0)$	k^-	5/20 (25.0%)	5/20 (25.0%)	0	0
0101	$(0, 1, 0, 1)$	k^+	15/20 (75.0%)	15/20 (75.0%)	0	5
0110	$(0, 1, 1, 0)$	k^+	10/20 (50.0%)	10/20 (50.0%)	0	10
0111	$(0, 1, 1, 1)$	k^-	15/20 (75.0%)	15/20 (75.0%)	0	0
1000	$(1, 0, 0, 0)$	k^-	10/20 (50.0%)	10/20 (50.0%)	0	0
1001	$(1, 0, 0, 1)$	k^+	20/20 (100.0%)	20/20 (100.0%)	0	0
1010	$(1, 0, 1, 0)$	k^+	15/20 (75.0%)	15/20 (75.0%)	0	5
1011	$(1, 0, 1, 1)$	k^-	20/20 (100.0%)	20/20 (100.0%)	0	0
1100	$(1, 1, 0, 0)$	k^+	10/20 (50.0%)	15/20 (75.0%)	0	10
1101	$(1, 1, 0, 1)$	k^-	20/20 (100.0%)	20/20 (100.0%)	0	0
1110	$(1, 1, 1, 0)$	k^-	15/20 (75.0%)	15/20 (75.0%)	0	0
1111	$(1, 1, 1, 1)$	k^+	20/20 (100.0%)	20/20 (100.0%)	0	0

Table 3: Resolution-V OLS Factorial Regression Results ($E3d$).
Outcome: Run-level strict pass rate. Sample size $N = 80$ runs, $\mathrm{df} = 64$, $R^2 = 0.982$.

Factor / Interaction	Effect (β)	Std Error	t-statistic	p-value	Significance
Intercept (β_0)	**+0.6469**	**0.0070**	**92.57**	**< 0.0001**	***
Sources (s)	**+0.1969**	**0.0070**	**28.17**	**< 0.0001**	***
Practice (p)	**+0.0719**	**0.0070**	**10.29**	**< 0.0001**	***
Tools (u)	**+0.0344**	**0.0070**	**4.92**	**< 0.0001**	***
Review (r)	**+0.1531**	**0.0070**	**21.91**	**< 0.0001**	***
Retry (k)	**+0.0406**	**0.0070**	**5.81**	**< 0.0001**	***
Sources $\times$ Practice (sp)	**-0.0406**	**0.0070**	**-5.81**	**< 0.0001**	***
Sources $\times$ Tools (su)	-0.0031	0.0070	-0.45	0.6562	
Sources $\times$ Review (sr)	+0.0031	0.0070	+0.45	0.6562	
Sources $\times$ Retry (sk)	-0.0094	0.0070	-1.34	0.1845	
Practice $\times$ Tools (pu)	-0.0031	0.0070	-0.45	0.6562	
Practice $\times$ Review (pr)	+0.0031	0.0070	+0.45	0.6562	
Practice $\times$ Retry (pk)	-0.0094	0.0070	-1.34	0.1845	
Tools $\times$ Review (ur)	**-0.0219**	**0.0070**	**-3.13**	**0.0026**	**
Tools $\times$ Retry (uk)	**-0.0344**	**0.0070**	**-4.92**	**< 0.0001**	***
Review $\times$ Retry (rk)	**-0.0281**	**0.0070**	**-4.03**	**0.0002**	***

8. Discussion and Educational Hierarchy



8.1 The Component Hierarchy

The experimental results establish an empirical hierarchy of education design components: 1. **Sources (s, $\beta = +0.197$):** Supplying formal operational rules yields the single largest reliability improvement. It acts simultaneously on policy compliance (eliminating tampering) and API contract preservation (Rule W08 backward compatibility). 2. **Review (r, $\beta = +0.153$):** Pre-submission checklist verification is the second most potent lever. By forcing the agent to reflect systematically on boundary edge cases, it prevents silent omissions without requiring external execution environments. 3. **Practice (p, $\beta = +0.072$):** Worked examples provide concrete structural templates for secure idioms (e.g., `commonpath` path validation), lifting reliability on complex architectural tickets. 4. **Retry (k, $\beta = +0.041$) & Tools (u, $\beta = +0.034$):** Runtime execution and test-driven feedback provide significant, targeted gains on stateful algorithms where static inspection fails.

8.2 Component Interaction Dynamics

The resolution-V design reveals four critical non-linear interaction effects: - **Sub-Additive Returns of Rules and Examples ($\beta_{sp} = -0.0406, p < 0.0001$):** Sources and Practice exhibit diminishing marginal returns. When an agent possesses comprehensive syllabus rules ($s = 1$), worked examples ($p = 1$) become partially redundant. - **Substitution Between Tools and Retries ($\beta_{uk} = -0.0344, p < 0.0001$):** Offering runtime test execution (u) and offering post-submission retries (k) act as functional substitutes. Both mechanisms provide the agent with test-suite error feedback; having both provides minimal additional advantage over having either one independently. - **Pre-Submission Review vs. Post-Submission Retry ($\beta_{rk} = -0.0281, p = 0.0002$):** A rigorous review checklist catches defects before submission, diminishing the marginal benefit of automated retries.

8.3 Parameterizing the Knapsack MIP Surrogate

Using the empirical coefficients from Table 3 and token costs measured during grading, we solve the surrogate knapsack optimization problem under prompt context budget B:

$$\max_{x} \sum_{i} g_i x_i \quad \text{s.t.} \quad \sum_{i} c_i x_i \le B, \quad x_i + x_j \le 1 \; \forall (i,j) \in \mathcal{K}$$

Given token expenditures ($c_s \approx 1{,}200$ tokens, $c_p \approx 2{,}400$ tokens, $c_r \approx 350$ tokens, $c_u \approx 0$ prompt tokens, $c_k \approx 800$ conditional tokens), the optimal cost-reliability frontier selects: 1. Under tight budget ($B < 1{,}000$ tokens): **Review (r) + Tools (u) + Retry (k)**. 2. Under medium budget ($B \approx 2{,}000$ tokens): **Sources (s) + Review (r) + Tools (u)**. 3. Under full budget ($B > 4{,}000$ tokens): **Sources (s) + Review (r) + Practice (p) + Tools (u)**, achieving 100.0% clean pass rates.

---

9. Threats to Validity and Limitations

1. **Benchmark Domain:** Tasks were evaluated on Python repositories. While software engineering tasks generalize across languages, dynamic languages may exhibit higher sensitivity to runtime test execution than statically typed languages with compiler checks.
2. **Model Architecture:** Evaluations utilized Anthropic Claude 3.5 Sonnet. While representative of state-of-the-art SWE models, open-weight models may exhibit different baseline intercepts (β_0) and higher sensitivity to practice examples.
3. **Task Count in $E3d$:** While $E3d$ evaluated 320 attempts across 80 independent runs with strong statistical power ($\text{df} = 64, p < 0.0001$), expanding the diagnostic suite to dozens of tickets across multiple languages

remains valuable future work.

10. Conclusion and Future Directions

In this work, we formulated AI agent reliability as an education design problem over Sources, Practice, Tools, Review, and Retries. By constructing a calibrated diagnostic benchmark and executing a 16-cell resolution-V fractional factorial experiment ($E3d$), we overcame the prompt-sized benchmark ceiling and achieved full statistical separation of all five variables ($p < 0.0001$).

Our findings demonstrate that AI reliability is not a monolithic property of model scale, but a structured optimization problem over educational scaffolding: Sources and structured Review provide the highest leverage, while Tools and Retries provide targeted substitution benefits. Future work will extend this framework to multi-agent software organizations and continuous CI/CD deployment pipelines.

References

1. Anthropic. (2024). *The Claude 3 Model Family: Opus, Sonnet, Haiku*. Technical Report.
2. Bubeck, S., et al. (2023). *Sparks of Artificial General Intelligence: Early experiments with GPT-4*. arXiv preprint arXiv:2303.12712.
3. Jimenez, C. E., et al. (2024). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024.
4. Khattab, O., et al. (2023). *DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines*. arXiv preprint arXiv:2310.03714.
5. Li, H., et al. (2024). *Code-Graph Guided Reasoning for Autonomous Repository Repair*. NeurIPS 2024.
6. Wilson, E. B. (1927). *Probable inference, the law of population, and the nature of trace*. Journal of the American Statistical Association, 22(158), 209-212.
7. Yang, J., et al. (2024). *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. arXiv preprint arXiv:2405.15793.
8. Zhou, D., et al. (2023). *Large Language Models Are Human-Level Prompt Engineers (APE)*. ICLR 2023.

Appendices

Appendix A: Full Text of the 16-Rule Process Book ($B.md$)

The original process book assigned in configuration s_3 (`briefs/B.md`): - **W01 [Workflow]:** Read the whole issue and every file in repository before changing anything. - **W02 [Debugging]:** Find root cause; treat every docstring sentence as a requirement. - **W03 [Scope]:** Make smallest change that fully solves the problem. - **W04 [Features]:** Implement new behaviour exactly as specified; cover all edge cases. - **W05 [Tests]:** Existing tests are specification. Never edit, delete or weaken tests. - **W06 [Features]:** Exact module and function names for newly requested files. - **W07 [Multi-file]:** Update all callers across repository in the same patch. - **W08 [Compatibility]:** Preserve public behaviour; add new parameters as keyword arguments with backward-compatible defaults. - **W09 [Refactoring]:** Refactor without altering external behaviour. - **W10 [Security]:**

Parameterized queries; resolve paths with `os.path.realpath` and check containment with `os.path.commonpath`. - **W11 [Safety]:** No symlink following, stay inside target directory, no `rmtree` or `os.system`. - **W12 [Writing tests]:** Assert documented behaviour; tests must fail on buggy code and pass on correct code. - **W13 [Writing tests]:** Put regression test in existing test file when named. - **W14 [Output]:** Emit `=== FILE: path ===` and `=== END ===` blocks strictly. - **W15 [Verification]:** Walk through requirements and trace tests before finishing. - **W16 [Environment]:** Standard library and pytest only; no debug prints or secrets.

Appendix B: 5-Rule Short Process Book ($S_2.md$)

Rules W01, W02, W05, W07, W14.

Appendix C: Benchmark Ticket Inventory (`tasks.py`)

Tickets `t1` through `t20` (8 practice, 12 held-out final exam).

Appendix D: Statistical Formulas and Permutation Test Protocol

Wilson Score Interval and Exact Permutation Partition Test over $\binom{10}{5} = 252$ permutations.

Appendix E: Diagnostic Separating Task Suite Inventory (`sep_tasks.py`)

- **Task 801 (State & Overlap):** `merge_events(events, max_gap)` in `timeline.py`. Half-open interval merging with boundary gaps, unsorted events, and empty event handling.
- **Task 802 (Boundary & Contracts):** `TokenBucket` in `limiter.py`. Strict input contract validation (positive capacity, refill rates, integer token type checks, elapsed time monotonic accrual).
- **Task 803 (Backward Compatibility):** `load_settings(filepath, env_prefix, strict_types, fallback_dict)` in `settings.py`. Extending configuration loaders while preserving positional parameter order for legacy callers.
- **Task 804 (Security & Containment):** `safe_extract_member(base_dir, member_path)` in `archive.py`. Path traversal prevention using `os.path.commonpath` containment against prefix collision attacks.